

ANÁLISE DOS PROTOCOLOS HTTP E HTTPS EM COMUNICAÇÃO COM ECOSSISTEMAS DE NUVEM

Guilherme Centenaro
João Henrique Antoniazzi
Rafael César Michalczuk

1. INTRODUÇÃO

O desenvolvimento de sistemas de comunicação na internet e a consolidação de infraestruturas distribuídas exigiram uma reestruturação profunda nos protocolos da camada de aplicação e de transporte. Originalmente projetada sob a premissa de interoperabilidade e entrega eficiente de pacotes, a suíte de protocolos TCP/IP não incorporou mecanismos nativos de segurança em seu design original.

Conforme descrito na perspectiva teórica clássica de James F. Kurose e Keith W. Ross na obra "Redes de Computadores e a Internet: Uma Abordagem Top-Down", o canal padrão proporcionado pelo protocolo TCP opera de maneira aberta, transmitindo todas as informações geradas pela camada de aplicação em formato de texto limpo (*cleartext*).

Com a transição de páginas estáticas para ecossistemas dinâmicos altamente integrados que interagem continuamente com soluções de banco de dados e computação em nuvem, essa ausência de segurança nativa passou a constituir uma vulnerabilidade severa para agentes maliciosos posicionados em nós intermediários da rede.

O protocolo HTTPS (Hypertext Transfer Protocol Secure) surgiu como uma salvaguarda computacional indispensável, aplicando criptografia híbrida para assegurar confidencialidade, integridade e autenticação.

Neste artigo, o grupo pretende abordar essa evolução, especificando o protocolo HTTP/HTTPS, assim como os mecanismos a ele atrelados, bem como a forma como interagem com as aplicações em questão de rede. Para caso de uso prático, evidencia-se o ecossistema de nuvem Firebase com Firestore, utilizado pelos autores no desenvolvimento do serviço para a disciplina de Projeto Integrador V, fato norteador da pesquisa.

2. FUNDAMENTOS DA CAMADA DE APLICAÇÃO E AS LIMITAÇÕES DO HTTP/1.1

O protocolo HTTP/1.1 funcionou como o principal motor de transferência de hipertexto por décadas, porém suas características estruturais impuseram gargalos significativos frente ao crescimento do volume de dados e scripts das aplicações modernas. Operando sobre conexões TCP persistentes, o HTTP/1.1 introduziu o mecanismo de *pipelining* para permitir o envio de múltiplas requisições sem esperar pelas respostas correspondentes.

Na prática, contudo, o *pipelining* apresentou sérias falhas de compatibilidade com proxies intermediários e dispositivos de rede, resultando frequentemente em seu desativamento, levando o protocolo a sofrer de forma persistente com o bloqueio de início de fila na camada de aplicação. Ou seja, se uma requisição inicial atrasar para ser processada ou transmitida pelo servidor, todas as requisições subsequentes na mesma conexão serão retidas na fila.

Para mitigar essa restrição, os navegadores modernos limitam-se a abrir o máximo de seis conexões TCP paralelas por domínio para o carregamento concorrente de recursos. Essa abordagem gera um custo computacional elevado, pois cada nova conexão exige o estabelecimento de um aperto de mão de três vias (*three-way handshake*) do TCP, introduzindo uma viagem de ida e volta de latência que prejudica o desempenho do sistema, especialmente em conexões com servidores distantes ou redes instáveis.

3. A EVOLUÇÃO TECNOLÓGICA DOS PROTOCOLOS HTTP/2 E HTTP/3

Para superar as limitações do HTTP/1.1, a evolução dos protocolos concentrou-se na otimização do transporte e na eliminação da latência. O HTTP/2 introduziu uma mudança estrutural ao implementar a multiplexação sobre uma única conexão TCP. Por meio de uma camada de enquadramento binário, as requisições e respostas são divididas em quadros e intercaladas de forma assíncrona em múltiplos fluxos (*streams*) independentes. Essa arquitetura permitiu abolir a limitação de seis conexões paralelas e reduziu o consumo de recursos computacionais no cliente e no servidor.

O HTTP/2 também introduziu a compressão de cabeçalhos via algoritmo

HPACK, reduzindo metadados redundantes como cookies volumosos, além do recurso de *Server Push*. No entanto, por depender de uma única conexão TCP subjacente, o HTTP/2 herdou a natureza sequencial desse protocolo de transporte.

O HTTP/3 soluciona este gargalo de transporte ao substituir o TCP pelo protocolo QUIC (Quick UDP Internet Connections), executado sobre a flexibilidade do User Datagram Protocol (UDP). No QUIC, o conceito de fluxos multiplexados é implementado diretamente na camada de transporte. Se um pacote associado a um determinado fluxo for perdido, apenas as informações daquele fluxo específico aguardam a retransmissão, enquanto os demais fluxos continuam transmitindo dados sem qualquer interrupção.

Adicionalmente, os algoritmos tradicionais de controle de congestionamento do TCP (como CUBIC e BBR) são lentos para atualizar por estarem acoplados ao núcleo do sistema operacional. O QUIC implementa o controle de congestionamento em espaço de usuário (*user space*), permitindo a rápida validação e adoção de modelos avançados (como CUBIC, BBR e RBBR) para maximizar a vazão e mitigar perdas em redes móveis.

A identificação das conexões no QUIC também foi desvinculada do tradicional quarteto de rede (IP de origem, porta de origem, IP de destino e porta de destino) através da introdução de um identificador exclusivo de conexão. Isso viabiliza o mecanismo de migração de conexão de forma transparente: quando um dispositivo móvel transita de uma rede Wi-Fi para dados celulares, a conexão persiste ininterruptamente com o mesmo identificador, evitando quedas de sessão e novas etapas de autenticação.

4. CRIPTOGRAFIA HÍBRIDA E A EVOLUÇÃO DAS CAMADAS SSL/TLS

Para garantir a segurança no tráfego de dados, o HTTPS utiliza uma arquitetura de criptografia híbrida que equilibra segurança matemática com desempenho computacional. Esse modelo combina a criptografia simétrica (onde uma mesma chave secreta cifra e decifra as mensagens) com a criptografia assimétrica ou de chave pública (onde cada entidade possui uma chave pública distribuída e uma chave privada mantida em sigilo absoluto).

A criptografia simétrica é extremamente rápida e adequada para cifrar grandes volumes de dados de requisições e respostas, porém apresenta o desafio

de distribuir a chave secreta entre cliente e servidor em um canal inseguro. A criptografia assimétrica soluciona esse problema, permitindo a transmissão segura da chave simétrica inicial e a autenticação do servidor através de certificados digitais validados por Autoridades Certificadoras confiáveis.

O estabelecimento de uma sessão HTTPS exige um protocolo de aperto de mão (*handshake*) que evoluiu de forma expressiva entre as versões 1.2 e 1.3 do TLS (Transport Layer Security). No TLS 1.2, o processo é longo e requer duas viagens completas de ida e volta antes do início da transferência segura de dados da camada de aplicação. Esse atraso degradava o desempenho de dispositivos móveis e sistemas sensíveis à latência.

O TLS 1.3 reduziu o tempo de conexão para apenas uma viagem de ida e volta, com o cliente enviando seus parâmetros de troca de chaves na primeira mensagem, permitindo que o servidor calcule a chave de sessão e envie seu certificado de forma criptografada na resposta direta. Em conexões recorrentes, o TLS 1.3 introduz o mecanismo de retomada rápida com latência zero, permitindo o envio de dados de aplicação imediatamente na primeira transmissão do cliente.

5. HTTP STRICT TRANSPORT SECURITY (HSTS): MECANISMOS DE PROTEÇÃO ATIVA E LIMITAÇÕES

Apesar das garantias de integridade oferecidas pelo HTTPS, a transição entre canais abertos e seguros apresenta vulnerabilidades críticas no primeiro contato do cliente. O método tradicional de escutar requisições na porta HTTP padrão (porta 80) e redirecioná-las (com códigos 301 ou 302) para a versão em HTTPS (porta 443) abre uma brecha para interceptações na primeira viagem de ida e volta do pacote.

Um atacante na rede pode interceptar a resposta HTTP não criptografada e reescrever o redirecionamento para manter o cliente em texto claro, realizando o sequestro da sessão (*SSL-stripping*) e coletando cookies, credenciais e dados sensíveis.

Para mitigar este risco, o IETF publicou a especificação RFC 6797, estabelecendo o HTTP Strict Transport Security (HSTS). O HSTS funciona como uma política de segurança declarada pelo servidor ao navegador do cliente por meio do cabeçalho de resposta HTTP Strict-Transport-Security. Quando o navegador do

cliente recebe esse cabeçalho sobre uma conexão HTTPS válida, ele registra localmente as diretivas de restrição e converte internamente qualquer tentativa futura de acesso HTTP para HTTPS antes de realizar qualquer envio para a rede.

Apesar de sua eficácia, o HSTS apresenta limites de segurança e privacidade, como incompatibilidade com ataques DNS, ataques no primeiro acesso, vulnerabilidades a ataques avançados do TLS e rastreamento e invasão de privacidade.

6. ARQUITETURA SERVERLESS E PROTOCOLOS DE INTEGRAÇÃO NO ECOSISTEMA FIREBASE

Na engenharia de sistemas em nuvem de alto desempenho baseados no ecossistema Firebase da Google, os conceitos teóricos de redes e segurança de tráfego de dados são aplicados de forma prática e escalável.

O tráfego reativo de dados para a leitura e escrita no banco de dados Cloud Firestore expõe diferentes canais de comunicação com base no ambiente do cliente. Em ambientes de servidor privilegiados como microsserviços rodando em instâncias Cloud Run ou Cloud Functions pelo Firebase Admin SDK, a comunicação ocorre nativamente através do gRPC (Remote Procedure Call).

O gRPC opera de forma exclusiva sobre canais multiplexados do HTTP/2, trafegando pacotes codificados usando Protocol Buffers (Protobuf) de alta compactação binária e type safety rígida.

Para estabelecer a comunicação em tempo real, o SDK do Firestore inicia o processo enviando uma requisição HTTP POST ao servidor, que gera um identificador único de sessão (SID). Esse SID passa a acompanhar todas as interações seguintes do cliente. A partir daí, qualquer operação de escrita ou envio de comandos (como atualizar documentos ou registrar novos ouvintes) é feita por meio de requisições POST independentes, que validam os dados no servidor e retornam de forma assíncrona.

6.1. Requisição de dados

Para receber as atualizações do banco de dados em tempo real, o SDK abre uma conexão persistente HTTP GET (semelhante ao Server-Sent Events) vinculada ao

mesmo SID. Essa conexão fica aberta por cerca de 1 minuto; sempre que há uma mudança nos dados, o servidor a envia imediatamente por esse canal. Assim que esse tempo expira, o servidor fecha a conexão de forma limpa e o SDK abre uma nova para manter o fluxo.

Se o usuário cancela as inscrições, um POST final encerra o SID. Caso a rede possua firewalls ou proxies que bloqueiem conexões longas, o SDK ativa automaticamente o Long-Polling: as conexões passam a expirar mais rápido (entre 15 e 30 segundos) e fecham logo após entregar qualquer mensagem, abrindo um novo canal em seguida para evitar bloqueios.

A evolução interna das bibliotecas cliente do Firebase demonstra uma preocupação constante com a estabilização e portabilidade desses canais. Na transição entre versões do SDK, observa-se alternância no motor de requisições de transporte.

Embora o uso de fluxos baseados na Fetch API ofereça maior aderência aos padrões de desenvolvimento modernos, a ocorrência de anomalias no controle interno de encerramento de conexões e fluxos bidirecionais sob certas condições levou o time de engenharia da Google a alternar transitoriamente as operações de streaming de dados para implementações clássicas baseadas em XHR (XMLHttpRequest), garantindo maior confiabilidade e consistência de rede multiplataforma.

6.2. Autenticação

A autenticação que governa esse ecossistema serverless apoia-se estritamente na passagem de tokens criptográficos estruturados no padrão JSON Web Token (JWT) e validados na origem do servidor. Ao efetuar o login por meio do SDK de autenticação do Firebase, o cliente obtém um token criptografado e assinado digitalmente pela Google.

Para chamadas de banco de dados por meio da API REST convencional, esse token deve ser injetado sob o cabeçalho HTTP clássico Authorization, de modo que o serviço de backend do Firestore consiga processar o token de assinatura e validar as permissões contra as regras de segurança (*Security Rules*) configuradas no console de gerenciamento.

Para o Firebase Realtime Database REST API, a validação aceita o token de

identificação via parâmetro *auth* de consulta, de modo a facilitar requisições de leitura de sistemas embarcados sem capacidade de injetar cabeçalhos complexos na chamada HTTP nativa.

7. CONCLUSÃO

A evolução da arquitetura de redes e dos protocolos da camada de aplicação e transporte reflete uma busca contínua pelo equilíbrio ideal entre segurança rigorosa e alta performance computacional. Conforme analisado, o modelo original da suíte TCP/IP, embora revolucionário em sua proposta de interoperabilidade, transferiu para as aplicações modernas o desafio de remediar a ausência de mecanismos nativos de proteção.

A resposta a essa vulnerabilidade consolidou o HTTPS e a criptografia híbrida como padrões indispensáveis, evoluindo de handshakes longos e custosos no TLS 1.2 para a eficiência de uma única viagem de ida e volta no TLS 1.3.

Paralelamente, a superação dos gargalos físicos e estruturais do HTTP/1.1 exigiu uma reengenharia profunda no transporte de dados. A introdução da multiplexação binária no HTTP/2 eliminou o limite mecânico de conexões paralelas por domínio, mas foi a quebra de paradigma do HTTP/3 ao substituir o TCP pelo QUIC sobre UDP que resolveu definitivamente o bloqueio de início de fila e introduziu a resiliência da migração transparente de conexões em ambientes móveis.

Complementarmente, mecanismos de defesa ativa como o HSTS demonstraram ser vitais para mitigar ataques de interceptação no primeiro contato (*SSL-stripping*), ainda que demandem atenção constante frente às suas limitações inerentes de DNS e privacidade.

Por fim, o estudo de caso do ecossistema Firebase (utilizado pelos integrantes do grupo ao longo do Projeto Integrador V) materializa como essa complexa pilha de protocolos opera na engenharia de software em nuvem moderna.

Aliado ao controle de acesso baseado em tokens JWT injetados em cabeçalhos ou parâmetros REST, o ecossistema prova que a segurança declarativa e a eficiência de rede não são excludentes, mas pilares codependentes que sustentam a escalabilidade e a confiabilidade dos sistemas distribuídos contemporâneos.

8. REFERÊNCIAS

Google (2026a). "Cloud Firestore Client Libraries". Google Cloud Platform. Disponível em: <https://firebase.google.com/docs/firestore/client/libraries>. Acesso em: 12 jun. 2026.

Google (2026b). "Configure custom headers - Firebase Hosting". Google Cloud Platform. Disponível em: <https://firebase.google.com/docs/hosting/full-config>. Acesso em: 12 jun. 2026.

Google (2026c). "Firebase Security Documentation". Google Cloud Platform. Disponível em: <https://firebase.google.com/docs/projects/security>. Acesso em: 12 jun. 2026.

Hodges, J., Jackson, C. and Barth, A. (2012). "RFC 6797: HTTP Strict Transport Security (HSTS)". Internet Engineering Task Force (IETF). Disponível em: <https://datatracker.ietf.org/doc/html/rfc6797>. Acesso em: 6 jun. 2026.

Jackson, C. and Barth, A. (2008). "ForcedHTTPS: Protecting High-Security Web Sites from Network Attacks". In: Proceedings of the 15th ACM Conference on Computer and Communications Security (CCS), Alexandria, USA, p. 1-14.

Kurose, J. F. and Ross, K. W. (2016). Redes de Computadores e a Internet: Uma Abordagem Top-Down. 7ª ed. Pearson Education do Brasil, São Paulo.

Rescorla, E. (2018). "RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3". Internet Engineering Task Force (IETF). Disponível em: <https://datatracker.ietf.org/doc/html/rfc8446>. Acesso em: 5 jun. 2026.